

Cotton Boll Sizing — Loss Function and RGB-D Plan

YOLOv11 supervised track, v3 annotation round

Rishi • April 30, 2026

1. Status and purpose of this document

This document responds to two of your asks from last Friday: (1) how I plan to incorporate and implement the various components of the loss function for the supervised cotton-boll model, and (2) how I plan to use the RealSense RGB-D data, specifically the depth maps and point clouds, in turning per-frame predictions into per-fruit size estimates. It also documents the state of the v3 annotation round Abhijeet just delivered, the gaps in that data, and the first model training run we are about to launch.

Where we are: the SAM 2 zero-shot pipeline is producing per-frame masks and rough per-fruit measurements end-to-end. The supervised YOLO track is set up, the v3 annotation is in hand, and we are ready to train. The v3 dataset implements the required parts of the annotation guideline (segmentation polygons + tip and base keypoints + boll_id linkage); a few attributes from the original spec are not in v3 and are listed in section 6 as the v4 ask.

2. Annotation data we have (v3 round)

Abhijeet exported each CVAT job in the native CVAT XML format, then ran a custom XML → COCO pass to merge the separately-labelled tip and base point shapes onto each cotton_boll polygon (matched by boll_id). This is the merged_dataset_v3/ folder we will train against; the raw CVAT XMLs are kept as a fallback. Two recordings are represented:

Recording	Train images	Val images	Boll instances	Notes
back_down	15	2	127 (113 tr + 14 val)	Looking back, downward angle
front_down	25	4	147 (129 tr + 18 val)	Looking forward, downward angle
Total	40	6	274	~6.85 bolls/frame avg

Per-instance schema (COCO categories + attributes):

- segmentation: pixel-accurate polygon of the visible boll surface.
- keypoints: [tip, base], COCO format with visibility flag in {0, 1, 2}.
- attributes.boll_id: the physical paper-tag number (fruit_id linkage).
- attributes.is_occluded, .occluded_by ∈ {none, leaf, stem, other_boll, flower, frame_edge_cutoff, other}.
- attributes.occlusion_pct ∈ [0, 1]: continuous estimate of the occluded area fraction.
- attributes.tip_visibility / base_visibility ∈ {visible, guessed, unknown}.

- `attributes.visible_parts` (optional polygon): the visible-only outline when `annotation_type = visible_parts`.
- `attributes.whole_estimate` (optional polygon): the full-extent outline including the occluded portion.

Three known issues we have already triaged (full v4 ask is in section 6):

- **Filename prefix mismatch.** COCO image entries store `file_name` as `src1_<frame>.jpg` but the files on disk are named `<frame>.jpg`. The `original_name` field has the correct value; our loader uses that.
- **Unplaced "visible" keypoints.** Of 274 instances, 95 (~35%) have a polygon attribute that says `tip_visibility` or `base_visibility = "visible"` but the corresponding keypoint coordinate is (0, 0) with `v=0`, because the annotator never placed the separate point shape on the image. The YOLO OKS keypoint loss zero-weights `v=0` keypoints, so this does not corrupt training, it just removes ~17% of available keypoint supervision.
- **Schema reductions.** v3 implements the required keypoints and a continuous `occlusion_pct`, but does not implement `visibility_fraction` (5-bin), `occlusion_type` (multi-label), `boll_stage`, `usable_for_size`, or the per-frame quality flags. We train against what is in v3 and request the missing fields in v4.

3. Loss function plan

The total training objective is a weighted sum of six terms. The first four are produced natively by Ultralytics-style YOLOv11 seg + pose models on the v3 supervision; the last two are produced by a small auxiliary classifier we train alongside, fed by the predicted boll bounding boxes. We chose this two-model decomposition rather than forking Ultralytics because (a) it keeps the geometry model — the part the downstream measurement code relies on — on the well-validated default training loop, and (b) it lets the auxiliary head be re-trained independently as more annotation comes in, without disturbing the seg/pose weights.

$$L_{total} = \lambda_{box} \cdot L_{box} + \lambda_{seg} \cdot L_{seg} + \lambda_{kp} \cdot L_{kp} + \lambda_{vis} \cdot L_{vis} + \lambda_{occ} \cdot L_{occ} + \lambda_{visfrac} \cdot L_{visfrac}$$

3.1 Geometry model (YOLOv11 seg + pose) — terms 1–4

Term	Supervision source	Loss form	Why this form
<code>L_box</code>	COCO bbox	CloU	CloU is the Ultralytics default; rotation- and aspect-aware, well-behaved at small overlaps where IoU/GIoU are flat.
<code>L_seg</code>	COCO segmentation polygon	BCE on prototype-coefficient mask + Dice on rasterised mask	YOLOv11-seg uses prototype masks; the BCE term drives pixel correctness, the Dice term penalises shape errors and is robust to the strong area imbalance (boll vs background).
<code>L_kp</code>	COCO	OKS (object-	OKS normalises pixel error by per-instance

Term	Supervision source	Loss form	Why this form
	keypoints (tip, base) with $v \in \{0,1,2\}$	keypoint similarity)	scale, so the same 5 px error costs more on a small distant boll than on a large near one — exactly the behaviour the downstream 3D-lifting step needs. $v=0$ keypoints contribute zero loss, which is what saves us on the 95 unplaced-but-marked-visible instances.
L_vis	tip_visibility, base_visibility $\in \{0,1,2\}$	Per-keypoint cross-entropy (3-class) on the pose head's visibility logit	Already produced by YOLO-pose natively; the loss form here matches Ultralytics. Lets the model output a per-keypoint visibility score at inference, which gates whether we trust the corresponding 3D-lifted coordinate downstream.

3.2 Auxiliary head (boll-crop classifier) — terms 5–6

The auxiliary head takes the predicted bounding box from the seg model, crops the boll region from the original image, resizes to 224×224, and feeds it to a small CNN (ResNet-18 pretrained on ImageNet, last block fine-tuned). The CNN has two output heads sharing the backbone: a 7-way softmax over `occluded_by` and a single sigmoid neuron for `occlusion_pct`. Both heads are trained jointly on the v3 train split with the same train/val partition as the geometry model.

Term	Supervision source	Loss form	Why this form
L_occ	<code>occluded_by</code> $\in \{7 \text{ classes}\}$	Categorical cross-entropy with class weights inversely proportional to frequency	Occluder class distribution is heavily skewed (e.g., <code>front_down</code> train has ~60% none, ~19% stem, ~17% leaf, the rest <5%). Class weighting prevents the head from collapsing to "none". The output is used at inference as a diagnostic, and as input to a per-frame trust score — <code>frame_edge_cutoff</code> and other_boll occlusions degrade size estimation more than leaf occlusions do.
L_visfrac	<code>occlusion_pct</code> $\in [0, 1]$	Smooth L1 (Huber) on a sigmoid-output scalar	Huber is robust to the long-tailed annotation noise on this attribute (Abhijeet computed it from polygon area ratios; outliers exist near small bolls). The predicted <code>occlusion_pct</code> feeds directly into the per-frame confidence weight that drives the

Term	Supervision source	Loss form	Why this form
			multi-frame size aggregation (see section 4.4).

3.3 Weighting and scheduling

Initial weights: $\lambda_{\text{box}} = 7.5$, $\lambda_{\text{seg}} = 4.0$, $\lambda_{\text{kp}} = 12.0$, $\lambda_{\text{vis}} = 0.5$, $\lambda_{\text{occ}} = 0.25$, $\lambda_{\text{visfrac}} = 0.5$. The first three are Ultralytics defaults for YOLOv11n-seg + pose. The auxiliary terms are deliberately scaled to contribute ~5–10% of the total gradient norm in the first few epochs so they do not destabilise the geometry head before mask quality has converged. We re-weight after epoch 30 by inspecting per-term gradient norms in TensorBoard and rescaling so each term contributes within an order of magnitude of the others.

Mosaic augmentation is on for the first 135 epochs and off for the last 15 — this is the Ultralytics `close_mosaic=15` idiom and matters because mosaic-augmented batches degrade mask edge quality at the very end of training. Vertical flip is disabled (would put base above tip) and horizontal flip is enabled with `flip_idx = [0, 1]` (tip and base do not swap under horizontal mirror).

3.4 Components from the spec that v3 does not yet support

Three signals from your annotation spec are not in v3 and are therefore not in this loss. The 5-bin `visibility_fraction` would let us convert `L_visfrac` from a regression to an ordinal classification with much cleaner gradients. The multi-label `occlusion_type` would turn `L_occ` from a 7-way single-label problem to a 7-way binary multi-label problem and would handle co-occurring occlusions (boll behind leaf and at frame edge) properly. The per-frame quality flags (`motion_blur`, `depth_failure`, `lighting_artifact`) and the per-fruit `usable_for_size` boolean would let us mask out frames at the loss level rather than relying on the model to infer unusability. The v4 ask in section 6 covers all three.

4. RGB-D and point cloud usage plan

The RealSense .bag files in `bags/` already carry the RGB stream and a per-pixel depth stream aligned to RGB intrinsics. The existing 5-stage SAM 2 pipeline extracts these to `work/handheld_*/{frames, depth}/`, plus per-fruit per-frame binary masks at `work/handheld_*/masks/<frame>/<tag>.png`. The supervised model's outputs slot into the same directory layout, so `pipeline/04_measure_bolls.py` is reused without changes.

4.1 Stage 1 — 2D prediction

For each frame the seg model produces a polygon and binary mask per detected boll, the pose model produces (`tip_xy`, `tip_v`) and (`base_xy`, `base_v`), and the auxiliary head produces a predicted `occluded_by` class and a predicted `occlusion_pct`. The four streams are joined on detection IoU so we have a single record per (frame, detection).

4.2 Stage 2 — 3D lifting via aligned depth

Each keypoint is projected into 3D in the camera frame: $(X, Y, Z) = Z \cdot K^{-1} \cdot (u, v, 1)^T$, where K is the RealSense colour intrinsics and Z is the median depth in a 5×5 window around the keypoint pixel. We

use the median rather than the single-pixel depth because individual pixels in the D4xx stream drop to zero at high frequency near edges and on dark surfaces; the median in a small window rejects those dropouts at minimal smoothing cost. If more than 60% of the window is invalid, the keypoint is flagged `depth_failure` and excluded from the 3D-derived metrics for that frame.

4.3 Stage 3 — per-frame size estimate

The boll long-axis length is the Euclidean distance between the lifted base and tip positions: $\|tip_3d - base_3dll$. The diameter is harder without the optional `width_left/width_right` keypoints — for v0 we estimate it as the largest 3D-lifted distance between mask boundary points perpendicular to the long axis, computed in a 2 cm slab around the boll midpoint. Once v4 adds the width keypoints we switch to the direct lifted `width_left / width_right` distance, which is more robust because it avoids the depth discontinuities at the mask boundary.

4.4 Stage 4 — pose-aware confidence and multi-frame aggregation

Define the boll's 3D long-axis direction as $u = (tip_3d - base_3d) / \|tip_3d - base_3d\|$ and the camera optical axis as $\hat{z}$. Compute $\cos \theta = |u \cdot \hat{z}|$. When $\cos \theta \approx 1$ the boll points almost straight at the camera and length is foreshortened (length unreliable, width still fine); when $\cos \theta \approx 0$ the boll is broadside to the camera and both axes are reliable. We translate this into per-measurement validity weights:

- $w_length(frame) = (1 - \cos^2\theta) \cdot (1 - occlusion_pct) \cdot detection_confidence$
- $w_width(frame) = (1 - occlusion_pct) \cdot detection_confidence$

Per-fruit length and diameter are then weighted means across all frames in which the boll appears, with frame i weighted by $w_length(i)$ and $w_width(i)$ respectively. Frames whose weight falls below a threshold (initially 0.1) are dropped. This is the confidence-weighted multi-frame aggregation you described — the predicted `occlusion_pct` from the auxiliary head is what closes the loop between the loss function (section 3.2) and the measurement code.

4.5 Point cloud as a global sanity check

Once per recording we collect every depth pixel inside any boll mask, lift them all to 3D, group by `boll_id`, and dump one .ply per fruit. For each fruit we fit a prolate spheroid (three axis-length parameters a, b, c via a least-squares fit on the 3D points, with the long axis constrained to align with the lifted base $\rightarrow$ tip vector). The fitted (length, diameter) are compared to the keypoint-derived estimates from stage 4.4 — when the two disagree by more than 15% on a fruit we flag that fruit as low-confidence and exclude it from the caliper comparison. The .ply files are also the most compelling visual artefact we can put in a slide: a 3D rendering of a tagged boll alongside its caliper measurement is much more convincing than a 2D bounding box.

4.6 Comparison against caliper ground truth

`metrics.csv` at the workspace root already has the per-fruit caliper `length_mm` and `diameter_mm` indexed by tag number. The model deliverable is a scatter plot of model length vs caliper length (and the same for diameter), with each point coloured by the dominant predicted `occluded_by` class for that fruit. This makes the model's failure modes visible at a glance: if leaf-occluded fruits cluster below the $y=x$ line, we know the diameter estimator is biased low for that occlusion regime, and we can focus the next round on it.

5. First model training run

The "are we heading in the right direction" run launches as soon as you sign off on this plan. It is two YOLOv11 trainings plus the auxiliary head, all on the v3 train split (40 images, 274 instances) with held-out val (6 images).

Component	Model	Settings	Notes
Segmentation	yolo11n-seg.pt	imgsz=1280, batch=4, epochs=150, patience=40, close_mosaic=15	Portrait images at 2988×5312 require $\text{imgsz} \geq 1280$ to keep small-boll detail; nano-size regularises well on a 40-image dataset.
Pose (2 kp)	yolo11n-pose.pt	imgsz=1280, batch=4, epochs=200, kpt_shape=[2, 3], flip_idx=[0,1]	Pose tends to need more epochs than seg. flip_idx is identity because tip/base do not swap on horizontal flip.
Aux head	ResNet-18 (ImageNet)	crop=224, batch=32, epochs=60, AdamW lr=1e-4, weight decay 1e-4	Trained on cropped boll bboxes from the v3 train images. Outputs occluded_by (7-class) + occlusion_pct (sigmoid).

"On the right track" thresholds (val):

- Seg: box mAP50 ≥ 0.85 , mask mAP50 ≥ 0.55 , mask mAP50-95 ≥ 0.30 .
- Pose: OKS mAP50 ≥ 0.40 , mean keypoint pixel error $\leq \sim 30$ px on a 5312-tall image ($\approx 0.6\%$ of long edge).
- Aux: occluded_by macro-F1 ≥ 0.45 , occlusion_pct MAE ≤ 0.15 .

These are below what we expect from a mature model — they are the floor for "the supervision is consistent and the model is learning the right thing", which is what this first run is designed to demonstrate. Deliverable for the meeting: a 6×3 visual grid of the val frames showing (a) ground-truth masks + keypoints, (b) predicted seg + pose, (c) predicted occluded_by label and occlusion_pct printed next to each detection. We will walk through the grid live.

6. v4 data round — what we are asking Abhijeet for

Below is the explicit list we will share with Abhijeet to make the next annotation round fully spec-compliant. Items are ordered by impact on the loss function.

v4 ask	Affected loss	Why it matters
Place tip / base point shapes wherever the polygon attribute says "visible"	L_kp, L_vis	~95 instances currently have an attribute saying "visible" but no point on the image. Recovers ~17% of available keypoint supervision at zero data cost.
Add visibility_fraction (5-bin: 1.0/0.75/0.5/0.25/0.0) per boll	L_visfrac	Lets us replace the noisy continuous occlusion_pct with a clean ordinal target. Big quality jump for the visfrac head, and the bins are easier for an annotator to assign reliably.
Add occlusion_type as multi-label (leaf, stem, other_boll, motion_blur, out_of_frame, self_occlusion, tag)	L_occ	Several real cases have two simultaneous occluders (boll behind a leaf and at the frame edge). v3 forces a single-class choice and loses information.
Add boll_stage $\in$ {unopened, partially_opened, opened}	(future) stage classifier	Not in v0 loss but needed for downstream agronomic analysis. Cheap to add.
Add usable_for_size (per-fruit per-frame boolean) and the per-frame quality flags (motion_blur, depth_failure, lighting_artifact)	frame masking before loss	Lets us drop bad frames before they contribute to L_seg / L_kp. Currently the model has to learn to ignore unusable frames, which is wasteful.
Optional: place midpoint, width_left, width_right keypoints on at least 50% of frames	L_kp (extended)	Required to switch the diameter estimator from mask-derived to keypoint-derived (much more robust to depth dropout). Add as soon as bandwidth allows; not blocking v4 sign-off.

7. Risks and open questions

Three things I want your read on before we lock in the v0 results:

- **Train/val sample size.** 6 val images is small enough that a single mis-detected frame moves mAP50 by ~0.05. We will report both the single-split numbers and a 5-fold CV over the 46 images so the variance is visible.
- **Diameter estimator without width keypoints.** The mask-perpendicular-distance estimator in section 4.3 is sensitive to depth dropout at the mask boundary. We expect diameter MAE to be 2–3× length MAE in v0 because of this. Adding the optional width keypoints in v4 should close most of the gap.
- **Auxiliary head data hunger.** 274 instances is on the low end for a 7-way classifier with a ~10M-parameter backbone. We will freeze the ResNet-18 backbone for the first ~20 epochs and only fine-tune the last block; if the head still over-fits we will downgrade to a 4-way collapsed taxonomy (none / leaf / stem / other) for v0 and reintroduce the full 7-way once v4 has more data.

8. Immediate next steps

In execution order, after sign-off on this plan:

- **1)** Write `merged_v3_to_yolo.py` to convert the v3 COCO into Ultralytics-format seg and pose datasets, handling the `src1_` filename strip.
- **2)** Kick off the seg + pose training (`runs/v3_seg/v0`, `runs/v3_pose/v0`). Wall clock estimate on a single consumer GPU: ~1.5 h each.
- **3)** Train the auxiliary classifier on cropped bboxes from the v3 train split.
- **4)** Generate the val visualization grid + caliper-comparison scatter plots for the meeting deck.
- **5)** Send the v4 ask list to Abhijeet so the next annotation round can start in parallel.

I will hold off on starting (1) until you are happy with this plan. Anything you want to cut, add, or reframe — the loss decomposition in section 3, the depth-window size in 4.2, the validity-weight formulae in 4.4, or the v4 priorities in section 6 — let me know and I will revise before we run a single epoch.